MYSX-256/512: A 256-bit Block Cipher with 512-bit Keys (v0.2-draft)

StringManolo

2026

Abstract

We present MYSX-256/512 v0.2-draft, an experimental 256-bit block cipher with 512-bit keys and 128 rounds. The design combines Feistel cells with Simon's F function, a Myszkowski byte transposition, word rotations, and an ARX key schedule.

Changes from v0.1-draft2. Two changes:

1. **Rotation schedule.** The consecutive schedule `(r mod 31)+1` of v0.1 is replaced by `rand_s9_45430` (31 explicit values in §2.8), selected by systematic search over 1M random 31-period schedules. On the metrics measured (differential at R=5, integral at R=5/6, linear at R=10/14 both directions, exact DMITM, weight-0 chain), the new schedule improves or matches v0.1 on **all 9 metrics** (6 improvements, 3 ties, 0 regressions; see §4.13).
2. **API v2.** A new packet format (VERSION=0x02, namespace `v2`) adds: 512-bit secrets processed through HKDF-SHA512 (eliminating the 256-bit HKDF-SHA256 bottleneck of v1), 512-bit MAC keys, HMAC-SHA512 with 64-byte tags, one key per message (32-byte random salt), authenticated header, Argon2id for password paths (with scrypt and PBKDF2-SHA512 fallbacks), and a version byte selecting the rotation calendar. The v1 API (`aeadEncrypt`, HKDF-SHA256, PBKDF2-SHA256, 32-byte tag) is kept unchanged for reading existing packets and is marked deprecated for new data.

Scope of this draft. The self-analysis of v0.1 (§3–§5 of the v0.1-draft2 paper: differential offset sweep, linear beam to 128 rounds, verified |corr|=1 chains, exact hulls at r=10,11,12, NIST SP 800-22, related-key audit) was performed with the consecutive schedule and is **not re-run for v0.2**. Sections that depend on the rotation schedule are explicitly marked **PENDING** with the command needed to resolve them. Sections that are properties of the round function or key schedule (exact LAT of F, mask propagation rules, key schedule audit) are inherited and are marked as such.

Measured for v0.2 (partial analysis, §4.12–4.13):

- Exact DMITM: **9 rounds** (v0.1: 17) — single-bit and multi-bit search over all offsets and both directions.
- Weight-0 chain: **5 rounds** (v0.1: 13) — exhaustive linear algebra over the whole left-half difference space.
- Integral balanced bits at R=5: **9/256** (v0.1: 21/256), at R=6: **4/256** (v0.1: 4/256), measured at r0=0, word0.
- MILP differential weight at R=5: **4** (v0.1: 0).
- Beam linear weights at R=10: forward=1, backward=3 (v0.1: 1 and 1); at R=14: forward=8, backward=11 (v0.1: 5 and 5). Higher weight = lower correlation = more secure.

- KAT: `KAT_v2.json` (v2 API vectors) and `KAT_v02.json` (block-only vectors) generated with the v0.2 schedule; both cross-verified against the Python model.

Not established for v0.2. The v0.1 differential and linear analyses at higher rounds, the truncated impossible differential (39 rounds for v0.1), the NIST SP 800-22 keystream battery, and the full 31-offset integral sweep are not re-run for the new schedule. The v0.1 cipher was not externally audited and neither is v0.2. **MYSX v0.2 is an experimental proposal, not a secure cipher; do not use it to protect real data.** For production: AES-256-GCM or XChaCha20-Poly1305.

Executive summary

MYSX-256/512 v0.2-draft is an experimental block cipher (256-bit block, 512-bit key, 128 rounds, SPN-like with 4 parallel Feistel cells per round). The v0.2 changes are a new rotation schedule and a new AEAD/packet format; the block cipher core (Feistel cells, F function, Myszkowski transposition, ARX key schedule) is unchanged.

Rotation schedule. The consecutive schedule `(r mod 31)+1` is replaced by `rand_s9_45430`. The schedule was selected by a systematic search: 1M random permutations of `0..30` were filtered by maximum weight-0 chain length at offsets 10 and 31, then by exact DMITM over the full left-half difference space; the 8 finalists were ranked by a multi-dimensional score including integral, linear, differential, and impossible analyses. The selected schedule is reproducible from `search_mysx_deep.py` and is documented as 31 explicit values in §2.8 (not as a seed/index of any language's PRNG).

API v2. A new packet format (VERSION=0x02) adds a version byte that selects the rotation calendar (no environment variable in production code), HKDF-SHA512 key derivation (which allows real 512-bit entropy when the caller supplies a 64-byte secret), HMAC-SHA512 tags, one key per message (32-byte salt), and Argon2id for password paths. The v1 API is kept for reading existing packets and is marked deprecated.

Measured for v0.2. For the block cipher with the new schedule: exact DMITM = 9 rounds (vs 17 for v0.1), weight-0 chain = 5 (vs 13), integral balanced bits at R=5 = 9 (vs 21), MILP differential at R=5 = 4 (vs 0), beam linear at R=14 both directions: 8 and 11 (vs 5 and 5; higher is better). Nine metrics measured, 6 improvements, 3 ties, 0 regressions. `KAT_v2.json` cross-verified between the JavaScript reference and the Python model.

Not measured for v0.2 (PENDING). The v0.1 analyses at higher rounds (differential offset sweep at R=14–24, linear beam to 128 rounds, exact hulls at r=10,11,12, NIST SP 800-22, related-key audit) were performed with the consecutive schedule. They are marked PENDING in their sections with the command needed to re-run. The v0.1 truncated impossible differential (39 rounds) was measured with the consecutive schedule only; the v0.2 truncated figure is PENDING.

Verdict. No lower bound over 128 rounds is established, no key-recovery attack has been attempted, and MYSX v0.2 has not been externally audited. It is released as v0.2-draft for public review with an explicit PENDING list. **Do not use MYSX to protect real data.**

1. Introduction

MYSX-256/512 is an experimental block cipher. It is offered as a public proposal for cryptanalysis, not as a production primitive. The author makes no claim of security beyond the observed results reported here, and explicitly welcomes external review.

1.1. Security objectives (and non-objectives)

Model. MYSX is analyzed in the single-key, chosen-plaintext model for confidentiality, combined with an encrypt-then-MAC AEAD wrapper for integrity (§2.9). The design does not claim security under related-key, chosen-ciphertext, or side-channel models; see §5 for related-key coverage and §9 for engineering notes.

Claimed security level. None formally. The paper reports observed upper bounds on single-trail weights (§3), a verified exact linear model up to a small number of rounds (§4), a beam search over that model in both directions for the v0.1 schedule (§4.10 of the v0.1 paper, PENDING for v0.2), integral distinguishers at R=5/6 for the v0.2 schedule (§4.12), exact DMITM at 9 rounds for the v0.2 schedule (§4.13), and a versioned rotation schedule selected by systematic search (§2.8). These are inputs to cryptanalysis, not proofs.

Reading thresholds. A differential trail of weight w needs about `2^w` chosen pairs; a linear trail of weight w has `|corr| = 2^-w` and needs about `2^(2w)` known texts. Therefore a `2^128`-text budget corresponds to a differential weight threshold of ≈128 but to a linear weight threshold of ≈64. **Differential and linear weights in this paper must not be compared directly; compare `w_diff` with `2·w_lin`.**

Non-objectives. MYSX v0.2 does not claim:

- post-quantum security beyond the trivial Grover bound (`2^256` quantum iterations for a uniformly random 512-bit secret processed through HKDF-SHA512; `2^128` for a 256-bit secret through HKDF-SHA256 in the v1 API, see §6);
- resistance to related-key attacks beyond the heuristic coverage of §5 (key schedule unchanged from v0.1; see note in §5);
- resistance to integral attacks beyond the R=5/6 distinguishers measured for v0.2 (offset-dependent; see §4.12);
- resistance to impossible-differential attacks beyond the exact DMITM = 9 measured for v0.2 (§4.13); the truncated figure for v0.1 is 39 rounds but for v0.2 it is PENDING;
- resistance to linear-hull effects beyond the single-trail weights measured for v0.2 (§4.12); hull-aware bounds are PENDING;
- constant-time execution in JavaScript (side channels out of scope).

1.2. Design rationale

MYSX combines four building blocks (unchanged from v0.1):

- **Simon's F** `F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`. Chosen for its studied diffusion/linear/differential properties in Simon and its simplicity (no S-boxes, only AND/XOR/rotl). It does not inherit Simon's security proofs; the surrounding structure is different. See §7 for what transfers and what does not.

- **Feistel cells over 32-bit word pairs.** Four parallel cells per round, then a byte transposition over the full 32-byte state, then word rotations. The result is classified as an SPN-like structure with Feistel cells, not as a "2-branch Feistel". See §2.1.
- **Myszkowski transposition** (§2.3.1). A fixed byte permutation derived from the keyword MYSZKOWS. The choice is a classical transposition with a keyword; it provides byte-level diffusion. A byte permutation optimized for diffusion (e.g., a structured permutation over the 4×8 byte grid) would strictly dominate it and is a candidate for a future version; see §9.
- **ARX key schedule** (§2.4). A bijective Feistel 8+8 update with modular addition, XOR, and rotations. Chosen for simplicity and to avoid S-boxes; analyzed in §5 (unchanged from v0.1).

Rotation schedule (v0.2). The consecutive schedule `(r mod 31)+1` of v0.1 is replaced by `rand_s9_45430` (31 explicit values in §2.8). The selection method and the criterion are documented in §2.8 and §4.13; the schedule was not chosen from a security argument on Myszkowski or on the round function, but from a multi-metric search. A comparison against 40 random byte permutations (with the same consecutive schedule) shows that MYSX v0.1 sits at the ~22nd percentile for exact DMITM: the consecutive schedule is not anomalous in the wrong direction — it is anomalous in the *good* direction (longest chains), which means it leaks the most structure to an attacker.

Parameter choices (256-bit block, 512-bit key, 128 rounds) are not derived from a formal argument. They were chosen to: provide a large block (avoid birthday-bound issues of 64-bit blocks); provide a large key (Grover leaves 256-bit security for a random 512-bit secret); and provide a large round count relative to Simon128/128 (68 rounds) as a precaution against unknown trails.

No claim is made that 128 rounds is sufficient. The round count is a parameter, not a security level.

1.3. Comparison with related designs

Design	Block	Key	Rounds	Structure	S-boxes
Simon128/128	128	128	68	Feistel	No
Speck128/256	128	256	34	ARX	No
ChaCha20	512	256	20	ARX	No
MYSX-256/512	256	512	128	SPN-like with Feistel cells	No

MYSX's closest sibling is Simon. The differences (block size, branch organization, transposition, word rotations, non-linear key schedule) are substantial enough that Simon cryptanalysis does not transfer directly (see §7).

Performance. MYSX is estimated to run at a few MB/s in JavaScript, versus hundreds of MB/s for AES-GCM via WebCrypto (AES-NI). MYSX is not competitive on performance; it is a research primitive.

1.4. Versioning and revision notes

Versioning. The reference implementation (`mysx.prod.js`) is labelled v0.2-experimental. This document is the v0.2-draft of the design. The v0.1-draft2 paper, tag `v0.1-draft2` in the repository, is the previous design (consecutive schedule, HKDF-SHA256 only, PBKDF2 password path, 32-byte tag). The two designs share the block cipher core (round function, F, transposition, key schedule) and differ in the rotation schedule and the AEAD/packet format.

What was re-measured for v0.2. Exact DMITM (§4.13), weight-0 chain (§4.13), integral at R=5/6 for r0=0 (§4.12), MILP differential at R=5 (§4.12), beam linear at R=10 and R=14 in both directions (§4.12), and KAT (`KAT_v2.json`, `KAT_v02.json`) cross-verified against the reference implementation (§2.11).

What was not re-measured for v0.2 (PENDING). Sections §3, §4.4–4.6, §4.9 offset sweep, §4.10 full beam, §4.11 NIST SP 800-22, and §5 related-key audit were measured for v0.1 and are marked PENDING for v0.2 with the command needed to resolve them. The v0.1 truncated impossible differential (39 rounds) was measured for the consecutive schedule; the v0.2 truncated figure is PENDING. Inherited sections are marked as such (properties of the round function or the key schedule, which do not depend on the rotation schedule).

2. Specification

2.1. Parameters

Parameter	Value
Block size	256 bits (8 words of 32 bits)
Key size	512 bits (16 words of 32 bits)
Rounds	128
Structure	SPN-like: 4 parallel Feistel cells + byte transposition
Non-linearity	`F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`
Diffusion	Myszkowski transposition (keyword MYSZKOWS)
Rotation	**`ROT_CAL[r mod 31]`** where `ROT_CAL` is the 31-value v0.2 schedule (§2.8)
Final whitening	XOR with 8 words from the key schedule

Structural note. MYSX is sometimes described as a “2-branch Feistel” by analogy with Simon. Strictly, each round applies four independent Feistel cells over 32-bit word pairs, then a byte transposition over the full 32-byte state, then word rotations. It is more accurately classified as an SPN with Feistel cells. The “Feistel” label is a convenience.

2.2. Notation

- `rotl(x, n)`: circular left rotation of x by n bits.
- ∧: bitwise AND.
- ⊕: bitwise XOR.
- `s[r]`: state at round r (`s[0]` = plaintext, `s[128]` = ciphertext).
- `s[r][i]`: word i (0..7) of the state at round r.
- `rk[r][i]`: subkey i (0..7) of round r.
- `rkFinal[i]`: word i (0..7) of the final whitening.
- `ROT_CAL[]`: the 31-value rotation schedule (§2.8).

2.3. Constants

```
CONST[0..7] = [
  0x243f6a88, 0x85a308d3, 0x13198a2e, 0x03707344,
  0xa4093822, 0x299f31d0, 0x082efa98, 0xec4e6c89
]
```

These are the fractional digits of π (same as Blowfish).

2.3.1. Myszkowski permutation Keyword: `MYSZKOWS` (8 letters, S repeated at positions 3 and 8).

Alphabetical order: K(5), M(1), O(6), S(3,8), W(7), Y(2), Z(4).

The byte permutation `MYZ_PERM[0..31]` is constructed as:

```
MYZ_PERM = [
   4,12,20,28,                 # K (column 4)
   0, 8,16,24,                 # M (column 0)
   5,13,21,29,                 # O (column 5)
   2, 7,10,15,18,23,26,31,     # S,S (columns 2 and 7, row-wise)
   6,14,22,30,                 # W (column 6)
   1, 9,17,25,                 # Y (column 1)
   3,11,19,27                  # Z (column 3)
]
```

The inverse permutation `MYZ_INV` satisfies `MYZ_INV[MYZ_PERM[i]] = i`.

Design note. The Myszkowski choice is a classical transposition with a keyword. It provides byte-level diffusion. A byte permutation optimized for diffusion (e.g., a structured permutation over the 4×8 byte grid) would strictly dominate it; the permutation is a candidate for replacement in a future version. See §9.

2.4. Key schedule

2.4.1. Initialization Input: key `K[0..63]` (64 bytes).

Initial state `s[0..15]` (16 words of 32 bits):

```
s[i] = big_endian_word(K[4i .. 4i+3])
```

2.4.2. Expansion For `r = 0 .. 127`:

```
spread = (r + 1) * 0x9E3779B9  (mod 2^32)

# Round subkeys
for i in 0..7:
    rk[r][i] = s[i] ⊕ s[(i+5) mod 16] ⊕ s[(i+11) mod 16]
               ⊕ CONST[(r+i) mod 8] ⊕ spread

# For i = 0..3: fold fk into rk
for i in 0..3:
    fk = s[(i+8) mod 16] ⊕ CONST[(r+i+3) mod 8]
    rk[r][i] = rk[r][i] ⊕ fk

# State update (Feistel 8+8, bijective)
L = s[0..7]
R = s[8..15]
for i in 0..7:
    a = R[i]
    b = R[(i+1) mod 8]
    c = R[(i+3) mod 8]
    F = (rotl((a + b) mod 2^32, 7) ⊕ rotl(c, 13) ⊕ CONST[i] ⊕ spread)
    s[i]     = R[i]
    s[i + 8] = L[i] ⊕ F
```

2.4.3. Final whitening After the 128 rounds:

```
for i in 0..7:
    rkFinal[i] = s[i] ⊕ s[(i+8) mod 16] ⊕ CONST[(128+i) mod 8]
```

2.5. Round function

Each round r transforms state `s[r]` into `s[r+1]` in four steps:

Step 1. AddRoundKey.

```
for i in 0..7:
    s[r][i] = s[r][i] ⊕ rk[r][i]
```

Step 2. Feistel cells. For `p = 0..3`:

```
L = s[r][p]
R = s[r][p + 4]
s_mid[p]     = R
s_mid[p + 4] = L ⊕ F(R)
```

where `F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`.

Step 3. Myszkowski transposition. Convert `s_mid[0..7]` to bytes (big-endian per word):

```
bytes[4i .. 4i+3] = big_endian(s_mid[i])
```

```
permuted[j] = bytes[MYZ_PERM[j]]  for j = 0..31
s_perm[i] = big_endian_word(permuted[4i .. 4i+3])
```

Step 4. Word rotation.

```
shift = ROT_CAL[r mod 31]        # v0.2: 31-value table from §2.8
for i in 0..7:
    s[r+1][i] = rotl(s_perm[i], shift)
```

2.6. Encryption

```
s[0] = plaintext  (as 8 big-endian words)
for r = 0..127:
    apply round r (section 2.5)
for i = 0..7:
    s[128][i] = s[128][i] ⊕ rkFinal[i]
ciphertext = s[128]  (as 32 big-endian bytes)
```

2.7. Decryption

Inverse of encryption:

```
s[128] = ciphertext
for i = 0..7:
    s[128][i] = s[128][i] ⊕ rkFinal[i]
for r = 127..0:
    apply inverse round r (inverse of section 2.5, in reverse order)
plaintext = s[0]
```

The inverse round consists of:

1. Inverse rotation: `rotl(word, 32 - shift)` where `shift = ROT_CAL[r mod 31]`.
2. Inverse transposition: `bytes[MYZ_PERM[j]] = permuted[j]`.
3. Inverse Feistel: `R = newL`, `L = newR ⊕ F(R)`.
4. XOR with `rk[r][i]`.

2.8. Rotation schedule (v0.2)

The schedule. The 31-value rotation schedule is:

```
ROT_CAL = [
  22,  5,  8,  7, 17, 10, 30,  1, 11, 26,
   6,  3, 12, 23, 25, 13, 28, 20, 21, 19,
  24,  2, 18,  9,  4, 15, 29, 27, 16, 14,
  31
]
```

`ROT_CAL` is a permutation of `1..31`. Round `r` uses `shift = ROT_CAL[r mod 31]`. The v0.1 consecutive schedule was `shift = (r mod 31) + 1`, i.e., `[1,2,3,...,31]`.

Selection method. The schedule was selected by systematic search over 1M random permutations of `0..30` (via `search_mysx_deep.py`, deterministic in the seed). Each candidate was filtered by:

1. Maximum weight-0 chain length at offset 10 ≤ 3,
2. Global maximum weight-0 chain length ≤ 5,
3. Exact DMITM over the full left-half difference space.

The finalists (8 candidates) were ranked by a multi-dimensional score including integral (balanced bits at R=5, R=6), linear (forward and backward single-trail weight at R=10, R=14, both directions), differential (MILP at R=5), impossible (exact DMITM), and weight-0 chain. The selected schedule is the Pareto-optimal among the 8 sampled.

Why the consecutive schedule was rejected. Across 3000 random schedules sampled without filtering, the consecutive schedule is in the worst ~2% by exact DMITM and in the worst ~0.2% by weight-0 chain. The consecutive schedule leaks more structure (longer chains) than 97–98% of random schedules. The cause is the regularity of the step: a consecutive sequence of shifts aligns with the period of the transposition in a way that lets the tracked bits remain in the "safe" half of the state for many rounds. The v0.2 schedule has irregular steps that break this alignment.

Comparison table (9 metrics, v0.2 vs v0.1).

Metric	v0.1 (`(r mod 31)+1`)	v0.2 (`rand_s9_45430`)	Result
dp5 (integral R=5, r0=0, word0)	21	**9**	improvement (−12)
dp6 (integral R=6, r0=0, word0)	4	4	tie
lin_fwd10 (single-trail weight, R=10)	1	1	tie
lin_bwd10 (single-trail weight, R=10)	1	**3**	improvement (+2)
lin_fwd14 (single-trail weight, R=14)	5	**8**	improvement (+3)
lin_bwd14 (single-trail weight, R=14)	5	**11**	improvement (+6)
DMITM exact	17	**9**	improvement (−8)
Weight-0 chain	13	**5**	improvement (−8)
MILP differential R=5	0	**4**	improvement (+4)

Directions: lower is better for dp5, dp6, DMITM, chain; **higher is better for**

lin_fwd10/14, lin_bwd10/14, MILP (higher weight = lower correlation = more secure). 6 improvements, 3 ties, 0 regressions.

Reproducibility. The schedule is documented as 31 explicit values above. It is reproducible from `search_mysx_deep.py --n 1000000 --seed 9` and selectable with `MYSX_ROT_CAL=22,5,8,...` in the analysis tools and with the CAL header byte `1` in the v2 API. `CALENDAR_v02.txt` contains the schedule as `MYSX_ROT_CAL=...`.

2.9. AEAD and packet format (v2)

Two packet formats coexist: v1 (VERSION=0x01, `aeadEncrypt/aeadDecrypt`, deprecated for new data) and v2 (VERSION=0x02, `v2.seal/v2.open`).

v2 packet.

```
VERSION(1)=0x02 | CAL(1) | KDF(1) | KDF_PARAMS(var) | SALT(32) | NONCE(16) | CT(n) | TAG(64
```

- `CAL`: rotation calendar selector. `0` = consecutive (v0.1-compatible block, not used by v2 by default), `1` = v0.2 schedule `rand_s9_45430`. Only table entries defined in the code are accepted; a packet with an unknown `CAL` is rejected.
- `KDF`: password/secret KDF selector. `0` = RAW (64-byte or 32-byte secret, direct HKDF). `1` = Argon2id. `2` = PBKDF2-SHA512. `3` = scrypt.
- `KDF_PARAMS`: parameters for the KDF, with bounds checked on parse:
  - Argon2id: `memKiB` (19456–1048576), `passes` (2–16), `lanes` (1–16). Defaults from RFC 9106 second recommended option: 65536, 3, 4.
  - PBKDF2-SHA512: `iterations` (210000–10000000).
  - scrypt: `logN` (15–20), `r` (8–16), `p` (1–16), memory bound 1 GiB.
- `SALT`: 32-byte random salt, one per message. Different per message means different key, eliminating nonce reuse in the key stream.
- `NONCE`: 16-byte random nonce, encrypted under the derived key.
- `CT`: ciphertext of the plaintext.
- `TAG`: HMAC-SHA512 over `HEADER | u64(len(AAD)) | AAD | u64(len(CT)) | CT`. Header includes VERSION, CAL, KDF, KDF_PARAMS, SALT, NONCE.

Key derivation (v2).

```
PRK     = HKDF-Extract-SHA512(salt = SALT, ikm)
info    = LP("MYSX-256/512 v2") | CAL | KDF | LP(context) | LP(label)
encKey  = HKDF-Expand(PRK, info(label="enc"), 64)
macKey  = HKDF-Expand(PRK, info(label="mac"), 64)
```

where `LP(x) = u16(len(x)) | x`. For KDF=RAW, `ikm = secret` (must be 64 or 32 bytes). For KDF=ARGON2ID/PBKDF2/SCRYPT, `ikm` is the password-derived key (64 bytes) followed by the optional keyfile (length-prefixed).

v2 API surface. `v2.seal(plaintext, creds, opts)`, `v2.open(packet, creds, opts)`, `v2.generateSecret()`, `v2.securityLevel(creds)`, `v2.setArgon2Provider(fn)`, `v2.argon2FromNoble(argon2idAsync)`, `v2.CAL`, `v2.KDF`, `v2.DEFAULTS`, `v2.LIMITS`. The `_salt` and `_nonce` options are internal (for KAT generation only) and must not be used in production.

2.10. Key derivation (v1, deprecated)

The v1 API is kept unchanged for backwards compatibility:

From a random secret (HKDF-SHA256):

```
encKey = HKDF-SHA256(secret, salt, info = context || ":enc", L = 64)
macKey = HKDF-SHA256(secret, salt, info = context || ":mac", L = 32)
```

From a password (PBKDF2-SHA256 + HKDF):

```
prk = PBKDF2-SHA256(password, salt, iterations = 600000, L = 32)
encKey = HKDF-SHA256(prk, salt, info = context || ":enc", L = 64)
macKey = HKDF-SHA256(prk, salt, info = context || ":mac", L = 32)
```

The HKDF-SHA256 bottleneck caps the entropy of a derived 512-bit `encKey` at 256 bits; the effective quantum security is `2^128`, not `2^256`. This is why v2 uses HKDF-SHA512 with a 64-byte secret. **Do not use v1 for new data.**

2.11. Test vectors

v1 KAT (`KAT.json`): generated with `generate_kat.py`, the consecutive calendar, HKDF-SHA256, PBKDF2-SHA256. Public digest: `5c9ca63d19e0f385eb0142bbe0337771d32a2526746` (SHA-256 over the full JSON). Internal fast-regression digest `KNOWN_KAT_DIGEST = 3a8084d0b7df7eb691cfb54494410a5222652c219a336ee58bde48a64524a8b3` (SHA-256 over 16 blocks with `key[i] = i`).

v0.2 block KAT (`KAT_v02.json`): generated with `generate_kat.py` under `MYSX_ROT_CAL=22,5,8,...` block-only vectors, no AEAD, no KDF.

v2 KAT (`KAT_v2.json`): generated with `generate_kat_v2.py`; contains: - `block_v02`: block cipher vectors with CAL=1, - `v2_raw`: v2.seal/open with a 64-byte secret, - `v2_password_argon2`: v2.seal/open with Argon2id, - `v2_password_scrypt`: v2.seal/open with scrypt, - `v2_password_pbkdf2`: v2.seal/open with PBKDF2-SHA512, - `v2_kdf`: direct derivation of encKey/macKey, - `v2_parse`: parse of a v2 header, - `security_level`: `securityLevel()` outputs for various creds, - `block_digest_cal0`, `block_digest_cal1`: SHA-256 over the 16 internal blocks for both calendars (the cal0 digest must equal `KNOWN_KAT_DIGEST`).

The block-cipher vectors of `KAT_v02.json` were cross-verified against `mysx_linear_model.py` (Python); the two independent implementations produce identical ciphertexts for the v0.2 schedule. A third-party implementation should reproduce `KAT_v02.json` block vectors, the v2 vectors of `KAT_v2.json`, and `KNOWN_KAT_DIGEST` for the consecutive calendar.

3. Differential cryptanalysis

Status: PENDING for v0.2. The differential analysis of the v0.1 paper (§3.1–§3.5) was performed with the consecutive schedule. Only the MILP differential at R=5 has been measured for v0.2 (§4.13). The following are the v0.1 results for reference and the commands needed to re-run for v0.2.

3.1. Model (inherited from v0.1)

Tool: `misc/mysx_milp2.py` (PuLP + CBC/HiGHS, with `--r0` rotation offset). AND model: `a = Δx ∨ Δy`, which counts a cost-1 event whenever at least one input is active. For an AND with independent inputs, this is the exact per-bit cost. The real approximations in the model are:

- The Markov-independence assumption does not hold: the two rotated inputs of F come from the same word x.
- The reported runs use a single input difference per offset; multi-bit input differences may yield lower weights.
- `gapRel = 0.2` and `timeLimit = 300–900` return feasible solutions (upper bounds), not certified optima.
- The model captures single trails, not differentials (clusters of trails with the same output difference).

3.2. Differential at r0=0 (v0.1 inherited)

At r0=0, the v0.1 results are:

Rounds	Input	Weight (upper bound)
5	w1b24	2
6	w1b24	6
7	w2b30	10
8	w2b30	18
12	w2b30	68
14	w2b30	114

PENDING for v0.2. Re-run with `MYSX_ROT_CAL=22,5,8,...` and `--solver highs --time-limit 180`. Expected to be higher than v0.1 because the weight-0 chain is 8 rounds shorter (§4.13).

3.3. Empirical r=4 at r0=0 (v0.1 inherited)

At r0=0, the r=4 trail (Δin = {w0b0}, MILP weight 8, Δout = {w0b4, w1b10, w1b16, w1b30, w6b4}) was verified with 8 fixed keys × 2^17 pairs each, 128-round cipher with final whitening disabled. Observed hit rate: `7.802 × 10^-3` over 1,048,576 pairs; expected `2^-8 = 3.906 × 10^-3`; ratio 2.00× (clustering factor).

PENDING for v0.2. Re-run with the v0.2 schedule via `misc/check_trail.js` after setting `MYSX_ROT_CAL`.

3.4. Differential offset sweep (v0.1 inherited)

At R=14, the v0.1 beam search found weights ranging from 2 at r0=10 to about 120 across offsets. The maximum observed at R=14 was 121 at r0=1, not 114 at r0=0. The r0=0 extrapolation of r≈15 is one point in a distribution.

PENDING for v0.2. The entire offset sweep must be re-run.

3.5. Differential weight-0 chains (v0.1 inherited)

In v0.1, a round has weight 0 iff the difference does not touch the right half (words 4..7). The free chain length depends on the rotation offset. The v0.1 free chain table (offset → max weight-0 rounds) was:

r0	0-6, 27-29	10	11	12	13	14	15	16	17	18
max rounds	3	**13**	12	11	10	9	8	7	6	5

The 13-round chain at r0=10 was verified against the real cipher with 8 fixed keys × 2^15 pairs each (262,144 pairs), 100% propagation.

v0.2 measured. The maximum weight-0 chain is **5 rounds** (see §4.13). The offset and the specific input/output masks of the v0.2 chains are PENDING. To measure:

```
MYSX_ROT_CAL=22,5,8,... python3 misc/diff_free_chain_table.py
```

4. Linear cryptanalysis

Status: partial for v0.2. The exact LAT of F (§4.2) and the mask propagation rules (§4.3) are inherited unchanged (properties of the round function, not of the schedule). The beam search (§4.10 of v0.1) is PENDING for v0.2. The R=10 and R=14 beam results for v0.2 are in §4.12. The hull checks (§4.6, §4.10.3–4.10.5) are PENDING.

4.1. Overview

An earlier version of the v0.1 paper reported only negative results three approximate linear MILP models all produced trails that failed empirical verification at w=32. Those models used per-cell AND activity as a proxy for correlation, which is not the correct structure of F's LAT. A correct model exists in closed form and is implemented in `cryptanalysis/models/linear/mysx_linear_model.py`. The model is machine-checked against the real cipher by `python3 mysx_linear_model.py self-test`.

Result in one line (inherited). The LAT of F is exactly `corr(b,g) = ± 2^-h(b)`, where h(b) is half the GF(2)-rank of a 32-cycle subgraph determined by b. Mask propagation is bit-exact; every trail carries an exact key-dependent sign. v0.1 verified `|corr|=1` chains over up to 9 consecutive rounds and exact linear-hull predictions at r=10,11,12.

v0.2 measured. With the new schedule: - Single-trail weights at R=10: forward=1, backward=3. - Single-trail weights at R=14: forward=8, backward=11. - See §4.12 for the full table.

4.2. Exact LAT of F (inherited from v0.1)

For output mask b and input mask g,

```
b.F(x) ⊕ g.x = Q_b(x) ⊕ l.x,
l = rotr(b,2) ⊕ g,
Q_b(x) = ⊕_j b_{j+1} x_j x_{j-7}
```

`Q_b` is a quadratic form whose graph `G_b` is a subgraph of the 32-cycle `j → j-7` (edge `(j, j-7)` present iff `b_{j+1} = 1`; a single cycle because gcd(7,32)=1). Let `A_b` be the GF(2) adjacency matrix of `G_b`. Then:

- `corr(b,g) ≠ 0` iff `l = A_b y` for some y. There are exactly `2^rank(A_b) = 4^h(b)` valid g.
- For each valid g, `corr(b,g) = (-1)^Q_b(y) · c_0(b)` with `|c_0(b)| = 2^-h(b)`, where `h(b) = rank(A_b)/2`.
- `h(b)` equals the maximum matching = minimum vertex cover of `G_b` (Kőnig). The only graph that is not a forest is `b = 0xFFFFFFFF` (full cycle), where rank=30 and h=15.
- Parseval holds: `Σ_g corr(b,g)^2 = 1`.

This is exact and machine-checked against brute-force LAT of F for random b of Hamming weight up to 8 (`selftest`).

4.3. Exact mask propagation (inherited from v0.1)

Forward step. For a round r, with input mask m on s[r] and cells p=0..3 (L=s[r][p], R=s[r][p+4]):

```
b_p = m[p]                                    # only words 0..3 decide the cost
g_p ∈ valid_gammas(b_p)                     # 4^h(b_p) choices, |corr|=2^-h(b_p)
n[p+4] = m[p]; n[p] = m[p+4] ⊕ g_p
m_next = lin_fwd(n, r)                          # byte transposition, then rotl
                                                 # by ROT_CAL[r mod 31] in v0.2
w_r = Σ_p h(m[p])
sign = (Π_p sign_p) · (-1)^(m_rk[r])
```

Backward step. For a round r, given `m_next` on s[r+1]:

```
n = lin_inv(m_next, r)                  # inverse rotation, then inverse transpose
b_p = n[p+4]; m[p] = n[p+4]; m[p+4] = n[p] ⊕ g_p
```

The selftest verifies that every forward edge is a backward edge with the same cost (325 edges checked), so the two searches enumerate the same set of trails.

Free-chain structure. A round has weight 0 iff words 0..3 of its input mask are zero; the round then acts as a pure bit permutation and the tracked bit never meets F. The longest v0.1 weight-0 chain is 9 consecutive rounds (windows r=12 and r=27 mod 31). The v0.2 free-chain table is PENDING (the measured v0.2 max chain is 5 rounds, §4.13).

4.4. Verified |corr| = 1 chains (v0.1 inherited, PENDING for v0.2)

v0.1. 9-round chains at r0=12 and r0=27 were verified against the real cipher: `|corr| = 1.00000 ± 6.9×10^-4` for three keys each, sign predicted exactly by the key-dependent formula.

PENDING for v0.2. The 9-round chains do not exist for the v0.2 schedule (max weight-0 chain is 5). The v0.2 verified chains, if any, are PENDING. Run:

```
MYSX_ROT_CAL=22,5,8,... python3 misc/verify_diff_trail.py
```

4.5. Beam search to 14 rounds (v0.1 inherited, partial for v0.2)

v0.1 at R=14 (beam 1200, single-bit starts): - r0=12 window: best weight 12 - r0=27 window: best weight 11

v0.2 at R=10 and R=14 (beam 2000, 31 offsets, both directions):

Metric	Weight	Direction
lin_fwd10 (min over offsets)	1	forward
lin_bwd10 (min over offsets)	3	backward
lin_fwd14 (min over offsets)	8	forward
lin_bwd14 (min over offsets)	11	backward

Higher weight = lower correlation = more secure. See §4.12 for the full 8-candidate comparison.

PENDING for v0.2. The full beam search to R=22, R=64, and R=128 (§4.10 of v0.1). To run:

```
MYSX_ROT_CAL=22,5,8,... python3 misc/mysx_linear_beam.py run \
  --offsets 0-30 --rounds 128 --beam 2000 --jobs 8 \
  --out beam_v02_full.json
MYSX_ROT_CAL=22,5,8,... python3 misc/mysx_linear_beam.py run \
  --offsets 0-30 --rounds 128 --beam 2000 --jobs 8 --direction backward \
  --out beam_v02_bwd.json
```

4.6. Hull verification at r=10,11,12 (v0.1 inherited, PENDING for v0.2)

v0.1. Exact signed hulls at r=10,11,12 matched the real cipher within sampling error (`|z| < 0.4`). See v0.1 §4.6.

PENDING for v0.2. Re-run `propagate()` from `mysx_linear_model.py` with the v0.2 schedule and the new beam trails.

4.7. What this establishes and what it does not (inherited)

Established (machine-checked by `selftest`):

- Exact LAT of F (valid set, magnitude, sign) against brute force.
- Exact per-round mask rule and key-sign tracking, in both directions (forward and backward edges match, 325 edges checked).
- Existence of key-independent-magnitude `|corr|=1` single-bit relations over up to 9 consecutive rounds **for the v0.1 schedule**.
- Exact signed linear-hull predictions at r=10,11,12 for the v0.1 schedule against the real cipher.

- The `mysx.prod.js` Python port reproduces `KNOWN_KAT_DIGEST`.
- The beam-search tool reproduces the §4.5 table exactly, its per-round masks agree with the real cipher wherever a trail is its own hull, and the selftest runs in both directions.

Not established:

- A lower bound on the best trail weight over 128 rounds.
- An exact 128-round hull.
- A hull-aware correlation beyond R=14.
- The cost of a key-recovery attack.
- Behaviour of multi-bit starts at larger R.

The weight-0 chains are not an attack. A 9-round (v0.1) or 5-round (v0.2) `|corr|=1` relation sits in a 128-round cipher. Standard linear cryptanalysis extrapolates a weight over the full length; here the trail weight over the first ~9 (v0.1) or ~5 (v0.2) rounds is 0 and the weight over the remaining rounds is unknown. The result is best read as: the exact machinery is in place and it agrees with the real cipher on small round counts, not as MYSX is broken.

4.8. Previous incorrect models and why they failed (inherited)

The v0.0 draft reported three approximate models, all with negative empirical verification. They used per-cell AND activity as a proxy for correlation, which is not the correct structure of F. The correct structure is the rank of the graph `A_b` above.

An instructive detail from the old work remains relevant: the empirical verification harness must use fixed keys per sample. With random keys per plaintext, the sign of every linear trail is key-dependent and the observed correlation of a genuinely nonzero trail collapses to zero.

4.9. Integral / division property (v0.1 inherited, partial for v0.2)

v0.1 results at r0=0:

Input	Active bits	Integral size	Rounds with balanced bits
bit 0 of word 0	1	2	10 (single-bit dependency, not an integral)
word 0	32	2^32	6
words 0 and 4	64	2^64	5

v0.1 results at r0=15: distinguishers extend to 10 rounds for both patterns (thin tail: 1 balanced bit at R=10, 0 at R=11).

v0.2 measured. At r0=0, word0:

Metric	v0.1	v0.2
dp5 (R=5, balanced bits)	21	**9**
dp6 (R=6, balanced bits)	4	4

Lower is better (fewer balanced bits = fewer filters for the attacker). The v0.2 schedule has 12 fewer balanced bits at R=5, and is tied at R=6.

PENDING for v0.2. The full 31-offset sweep and the r0=15 peak. To run:

```
MYSX_ROT_CAL=22,5,8,... python3 misc/mysx_division_property_v2.py \
  --rounds 5-10 --active word0 --r0-sweep --solver highs
```

4.10. Full beam search, 128 rounds (v0.1 inherited, PENDING for v0.2)

v0.1 results (forward + backward + merged, all 31 offsets, up to 128 rounds):

R	Best weight	Offset	Direction
12	2	20	backward
14	5	20	backward
20	38	20	backward
22	63	23	forward
64	786	27	forward
128	2248	3	backward

Largest R with weight < 64 (≈2^128 texts): 22 (tie with the threshold). Weight grows ~12–21 per round from R=22 to R=112.

PENDING for v0.2. Re-run with the v0.2 schedule via the beam tool (see §4.5 for the command). Expected to be **higher** than v0.1 because the weight-0 chain is 8 rounds shorter.

4.11. Statistical randomness of the CTR keystream (v0.1 inherited, PENDING for v0.2)

v0.1 result. The CTR keystream passed the full NIST SP 800-22 battery (100 sequences × 10^6 bits, fresh key and nonce per sequence, no failures, all margins inside the NIST acceptance region). TestU01 BigCrush and dieharder were not run.

PENDING for v0.2. Re-run with the v0.2 schedule:

```
MYSX_ROT_CAL=22,5,8,... MYSX_ENABLE_TEST=1 node misc/gen_nist_input.js
cat misc/nist_input/data* > misc/nist_input/all_data.bin
./assess 1000000   # in STS 2.1.2
```

4.12. Impossible differentials (measured for v0.2)

Exact DMITM (single-bit and multi-bit).

A differential round has weight 0 iff the right half of the state difference is zero; the round is then the linear layer only (Myszkowski + word rotation), because `ΔR=0 ⇒ ΔF(R)=0` and the Feistel cell reduces to a swap. Weight-0 chains of up to 5 rounds exist for the v0.2 schedule (vs 13 for v0.1, see §4.13). A miss-in-the-middle search over forward and backward chains gives the exact DMITM length.

v0.2 result. Exact DMITM = **9 rounds**. This is 8 rounds shorter than the v0.1 value (17 rounds, §3.5 of the v0.1 paper). The impossibility is structural: the forward and backward chains reach the same intermediate state with contradictory differences, and the contradiction is checked bit-by-bit in the truncated-propagation sense (bits classified as known-0, known-1, unknown; the Simon F only marks a bit unknown if it depends on its AND).

Verification. The premise that each chain is deterministic in the real cipher is verified empirically: forward and backward chains give 100% propagation over 18,000 pairs × 6 keys per side (including degenerate keys 0x00… and 0xFF…) in the v0.1 truncated case. The v0.2 exact DMITM is a structural consequence of the weight-0 chains (which are themselves proven by exhaustive linear algebra over the whole left-half difference space); the empirical test is a sanity check and an implementation validation, not a statistical proof.

Truncated impossible differential (PENDING for v0.2).

For v0.1, the truncated impossible differential reaches **39 rounds** (propagating bits classified as known-0, known-1, unknown from both sides and looking for a bit-level contradiction). For v0.2, this number is **PENDING**: it requires re-running `dmitm_trunc.py` with `MYSX_ROT_CAL=22,5,8,...` and `--n_bits 256` (the v0.1 numbers used the same script with the consecutive schedule). Expected to be shorter than 39, proportional to the reduction in weight-0 chain length (13 → 5 rounds).

Single-bit and multi-bit variants (PENDING for v0.2).

For v0.1, single-bit and multi-bit impositives at 15 and 17 rounds were verified against the real cipher. For v0.2, the exact DMITM = 9 is a multi-bit result over the full left-half difference space; the analogous single-bit and multi-bit truncated-propagated variants are PENDING. To run:

```
MYSX_ROT_CAL=22,5,8,... python3 cryptanalysis/DMITM/dmitm_verify.py exact
MYSX_ROT_CAL=22,5,8,... python3 cryptanalysis/DMITM/dmitm_trunc.py \
  --cal_file top50_fixed.json --cal_idx 3 --max_fwd 24 --max_bwd 24 --n_bits 256
```

Structural origin.

The trace shows that the v0.2 chains visit the same word at the meeting point but with different bits, because the accumulated rotations differ. The cause of long v0.1 chains was the **consecutive** rotation schedule (step 1) aligning with the period of the transposition; the v0.2 schedule has irregular steps that break this alignment. This is a property of the schedule, not of Myszkowski specifically (a random permutation with the same consecutive schedule gives a mean of 13 weight-0 chain length over 40 samples, and a mean of ~21.4 exact DMITM; the consecutive schedule is in the ~2% best case for DMITM and ~0.2% for chain over 3000 random schedules).

Not established for v0.2.

- The truncated impossible differential (§4.12 above, 39 rounds for v0.1, PENDING for v0.2).
- The single-bit and multi-bit truncated-propagated variants (PENDING).
- No plaintext-to-ciphertext distinguisher, no key recovery.
- No related-key impossible analysis.

4.13. Rotation schedule selection (v0.2)

Method. 1M random permutations of `0..30` were sampled (`search_mysx_deep.py`). Each was filtered by (a) weight-0 chain at offset 10 ≤ 3, (b) global weight-0 chain ≤ 5, (c) exact DMITM over the full left-half difference space. 8 candidates reached exact DMITM = 9. The 8 finalists were ranked by a multi-dimensional score including integral, linear (forward and backward), differential, and impossible analyses.

Baselines. The v0.1 consecutive schedule `(r mod 31)+1`, the schedule `(2r mod 31)+1` (smallest non-consecutive schedule), and the schedule `(22r mod 31)+1` (best affine schedule found by structured search over 30 candidate values of k).

Results (8 candidates × 9 metrics).

Schedule	dp5↓	dp6↓	linF10↑	linB10↑	linF14↑	linB14↑	DMITM↓	chain↓	milp5↑
`(r mod 31)+1` (v0.1)	21	4	1	1	5	5	17	13	0
`(2r mod 31)+1`	28	3	2	2	4	5	12	9	0
`(22r mod 31)+1`	29	10	3	3	4	14	10	6	8
`rand_s9_45430` (v0.2)	**9**	**4**	**1**	**3**	**8**	**11**	**9**	**5**	**4**
`rand_s9_30719`	30	12	2	4	13	16	9	5	10
`rand_s6_14172`	11	6	3	5	12	17	9	5	2
`rand_s8_24902`	24	11	0	0	7	7	9	5	4
`rand_s5_24657`	24	9	3	3	14	12	9	5	6

Directions: dp5, dp6, DMITM, chain = lower is better; linF10, linB10, linF14, linB14, milp5 = higher is better (higher weight = lower correlation = more secure).

Selected schedule: `rand_s9_45430` (the 31 explicit values in §2.8).

Against v0.1:

- dp5: 21 → 9 (improvement)
- dp6: 4 → 4 (tie)
- linF10: 1 → 1 (tie)
- linB10: 1 → 3 (improvement)

- linF14: 5 → 8 (improvement)
- linB14: 5 → 11 (improvement)
- DMITM: 17 → 9 (improvement)
- chain: 13 → 5 (improvement)
- milp5: 0 → 4 (improvement)

6 improvements, 3 ties, 0 regressions.

A false positive of R=4 screening. `rand_s9_33719` was top-ranked by R=4 screening (with dp4=42 and milp5=10). At R=6 its integral distinguisher is worse than v0.1 (12 vs 4 balanced bits), and its linear forward weight at R=14 is 13 vs 5. **Screening at R≤4 is not sufficient**: candidates must be evaluated at R=5-6 for integral and R=10-14 for linear before selection.

Why Myszkowski is not the cause. The rotation schedule is what determines chain length. A control with 40 random byte permutations and the same consecutive schedule gives a mean exact DMITM of 21.4 (range 10–34); MYSX v0.1 sits at ~22nd percentile. Changing the consecutive schedule to `(k·r mod 31)+1` for k∈{2,3,5,7} drops the maximum chain to 6–9 rounds. The factor that controls the long weight-0 chains is the **step of 1** in the rotation schedule, not the Myszkowski permutation.

Reproducibility. `CALENDAR_v02.txt` contains `MYSX_ROT_CAL=22,5,8,...`. `search_mysx_deep.py --n 1000000 --seed 9` regenerates the search. `cryptanalysis/DMITM/wf2_deep_summary.json` contains the raw 8-candidate × 9-metric table.

5. Related-key cryptanalysis (inherited from v0.1)

Note. The key schedule is unchanged from v0.1. The 22 structural and empirical tests of v0.1 §5 apply to v0.2 with the same key schedule. The rotation schedule does not affect the key schedule. The results are inherited; the specific key schedule tests (`misc/keyschedule_test.js`, `misc/keyschedule_test_extended.js`) should be re-run on the v0.2 branch to confirm reproducibility.

v0.1 results (inherited). No persistent `Δrk[r] = 0` for r ≥ 1 in any test. Beam search score: 139 (ideal 128). No weak keys, cycles, or fixed points. Slow diffusion at r=1–4 (expected: one Feistel round per cipher iteration). Worst bit (minimum weight at r=1): bit 160 (word 5, bit 0), 1-bit difference. Consistent across 5 base keys (structural, not noise).

Limitations (inherited). These are heuristic tests, not a proof. No targeted MILP/SAT related-key model was run; adaptive related keys, biclique attacks, and algebraic attacks on the key schedule are not covered. MYSX's related-key security is unknown.

Round-key recovery. The only non-linear operation of the key schedule is the 8 modular additions per step; everything else is XOR and rotation. The schedule is bijective, so any 16 consecutive state words determine the master key. How many round keys must be known for a SAT solver to recover the master key (estimate: about three consecutive `rk[r]`) has not been measured.

6. Generic attacks (inherited, updated for v2 API)

Attack	Complexity	Verdict
Classical brute force	2^512 operations (uniformly random 512-bit secret)	Infeasible
Grover (quantum)	2^256 iterations (uniformly random 512-bit secret)	Infeasible with foreseeable technology

Note on Grover (v2 API). The v2 API accepts a 64-byte secret processed through HKDF-SHA512. With 64 bytes of uniformly random entropy from the caller, the effective quantum security is `2^256` (Grover) for one target. With 1,000,000 logical qubits at 1 ns/iteration, the time would be `~10^60` years. This is a theoretical estimate; no post-quantum analysis beyond Grover has been done.

Note on the v1 API (deprecated). The v1 API (`deriveAeadKeys`, `deriveAeadKeysFromPassword`) starts from HKDF-Extract with HMAC-SHA256, whose output is 256 bits. The entropy of a derived 512-bit `encKey` is at most 256 bits. The Grover figure for v1 is `2^128`, not `2^256`. The v2 API replaces HKDF-SHA256 with HKDF-SHA512 and a 64-byte secret, recovering the `2^256` quantum bound. The `2^512` and `2^256` classical/Grover figures apply only when the caller supplies a uniformly random 64-byte secret directly to `v2.seal({secret})`.

Note on the 32-byte secret (v2). If the caller uses a 32-byte secret with `v2.seal({secret})`, the effective classical security is 256 bits and the Grover figure is `2^128`. `v2.securityLevel` returns this distinction explicitly. For password paths with a keyfile of less than 64 bytes, security is limited by password entropy.

7. Simon cryptanalysis: what transfers and what does not

MYSX uses Simon's F. It is not Simon. The differences:

- Block size and branch organization: MYSX is an SPN-like structure with four parallel Feistel cells over 32-bit word pairs (§2.1). Simon is a classical 2-branch Feistel over n-bit halves.
- Four parallel F per round (Simon uses 1).
- Myszkowski byte transposition (Simon has none).
- Round-dependent word rotations (Simon has none).
- Non-linear ARX key schedule (Simon's is linear).
- 128 rounds vs 68.

What transfers. The exact LAT of F — its closed form `|corr| = 2^-h(b)`, its validity condition, and its sign structure — is a property of the function and therefore applies to MYSX regardless of the surrounding structure (§4.2 is the transfer). That is the correct starting point for MYSX-specific linear cryptanalysis.

What does not transfer. Key recovery from Simon's linear key schedule does not transfer: MYSX's key schedule is non-linear. Linear hull attacks against Simon are

highly structured around Simon's Feistel-branch and its round-reduced behaviour; the MYSX hull is a different object and must be computed from the MYSX round model. Integral distinguishers against Simon rely on Simon's particular 2-branch Feistel structure and its key schedule; the MYSX integral distinguishers are considerably shorter (relative to total rounds) and their relationship to Simon's is one of normalized coverage, not direct transfer.

Relevant literature on Simon (cited for context). Chen-Wang ("Improved Linear Hull Attack on Round-Reduced Simon with Dynamic Key-guessing Techniques", FSE 2016). Shi-Hu-Sun-Song-Qiao-Ma ("Improved Linear (hull) Cryptanalysis of Round-reduced Versions of SIMON", ePrint 2014/973). Biryukov-Roy-Velichkov ("Differential Analysis of Block Ciphers SIMON and SPECK", FSE 2014). Todo-Morii ("Bit-Based Division Property and Application to Simon Family", FSE 2016). Xiang-Zhang-Bao-Lin (ASIACRYPT 2016). Kölbl-Leander-Tiessen ("Observations on the SIMON block cipher family", CRYPTO 2015).

PENDING for v0.2. The Simon comparison in v0.1 used the consecutive schedule for the MYSX figures. Re-run the MYSX linear hull, integral, and impossible-differential comparisons with the v0.2 schedule before publishing a normalized coverage comparison.

8. Analysis limitations

Differential (PENDING for v0.2). v0.1 upper bounds over 31 rotation offsets at r0=0 and the free-chain table are inherited as reference only. For v0.2, only the MILP differential at R=5 has been measured (weight 4 vs 0 for v0.1). The v0.2 offset sweep and R=14+ MILP are PENDING.

Linear (partial for v0.2). An exact model of F is established and matches the real cipher on small round counts for v0.1. For v0.2, the beam search has measured R=10 and R=14 both directions (weights 1, 3, 8, 11). The full 128-round beam, exact hulls, and hull checks are PENDING. Higher is better for linear weight (lower correlation = more secure).

Integral (partial for v0.2). Measured at r0=0, word0: dp5=9, dp6=4 (v0.1: 21, 4). The full 31-offset sweep at r0=15 and R=8-10 tails are PENDING.

Impossible differential (partial for v0.2). Exact DMITM = 9 rounds (v0.1: 17). Truncated impossible differential is PENDING for v0.2 (39 for v0.1); single-bit and multi-bit truncated variants are PENDING.

Related-key (inherited). Key schedule unchanged; v0.1 results (heuristic) apply. Not re-run on v0.2 branch.

Other. No boomerang, algebraic, or side-channel analysis. No post-quantum analysis beyond the trivial Grover bound. No independent public cryptanalysis. Functional tests (`mysx.test.js`, 17 tests) cover round-trip, AEAD negatives, nonce policy, PBKDF2, avalanche. They do not prove security.

9. Engineering notes

9.1. API surface

Public API (v2, recommended for new data):

- `v2.seal(plaintext, creds, opts)` — encrypt.
- `v2.open(packet, creds, opts)` — decrypt.
- `v2.generateSecret()` — returns a 64-byte Uint8Array for the caller to store as the user's secret.
- `v2.securityLevel(creds)` — declared nominal security for the creds.
- `v2.setArgon2Provider(fn)` — register an Argon2id implementation.
- `v2.argon2FromNoble(argon2idAsync)` — adapter for `@noble/hashes`.
- `v2.CAL`, `v2.KDF`, `v2.DEFAULTS`, `v2.LIMITS` — constants.

Public API (v1, deprecated for new data):

- `aeadEncrypt(plaintext, key, macKey, opts)` — v1 encrypt.
- `aeadDecrypt(packet, key, macKey, opts)` — v1 decrypt.
- `deriveAeadKeys(secret, opts)` — v1 HKDF-SHA256.
- `deriveAeadKeysFromPassword(password, salt, opts)` — v1 PBKDF2-SHA256.
- `NonceGuard` — v1 nonce reuse guard.
- `CONFIG` — constants.

Test API: disabled by default. Enabled only when `MYSX_ENABLE_TEST=1` is set in the process before requiring the module. There is no code-level toggle.

9.2. Migration from v1 to v2

- v1 packets (`0x01...`) are still decryptable with `aeadDecrypt`.
- v2 packets (`0x02...`) are created by `v2.seal` and are not decryptable by `aeadDecrypt`. `v2.open` rejects v1 packets.
- **Do not use `aeadEncrypt` for new data.** It uses HKDF-SHA256, which caps the quantum security at `2^128`. Use `v2.seal` with a 64-byte secret for `2^256`.
- **Do not pass `_salt`/`_nonce` to `v2.seal` in production.** They are internal parameters for deterministic KAT generation. Their presence in the API is a known hygiene issue; a future version should gate them behind an explicit `internal: true` flag.

9.3. NonceGuard (v1, deprecated)

NonceGuard is a best-effort defense against nonce reuse in the v1 API. It counts messages per key (default `2^32`), detects recent nonce reuse (window of 1024 per key), caps the number of tracked keys (default 1024, LRU eviction), identifies keys by a non-cryptographic 64-bit hash, throttles `onSave` calls (default: once every 100 `check()` calls), does not persist across restarts by itself, does not share state across processes or devices.

The v2 API does not need NonceGuard: every message uses a fresh 32-byte salt and a fresh 16-byte nonce, so accidental (key, nonce) reuse has probability `~2^-384` per message pair.

9.4. Type validation

All public functions validate that byte inputs are `Uint8Array` of the expected length (where applicable). Passing a string as `aad` or `plaintext` throws instead of silently coercing to array-like behavior.

9.5. KAT digests

Three KAT files:

- `KAT.json` (v0.1): `block`, `aead`, `kdf`, generated with the consecutive schedule, HKDF-SHA256, PBKDF2-SHA256. Public digest: `5c9ca63d19e0f385eb0142bbe0337771d32a252`
- `KAT_v02.json` (v0.2 block only): `block` vectors with the v0.2 schedule.
- `KAT_v2.json` (v0.2 API): `block_v02`, `v2_raw`, `v2_password_*`, `v2_kdf`, `v2_parse`, `security_level`, `block_digest_cal0`, `block_digest_cal1`.

`KNOWN_KAT_DIGEST = 3a8084d0b7df7eb691cfb54494410a5222652c219a336ee58bde48a64524a8b3` is the v0.1 fast-regression digest (16 blocks with `key[i] = i`). The `block_digest_cal0` in `KAT_v2.json` must equal `KNOWN_KAT_DIGEST`; if it does not, the v1 path is broken.

A third-party implementation should target: - `KAT.json` (v0.1 vectors), - `KAT_v02.json` block vectors (v0.2 block), - `KAT_v2.json` v2 vectors (v0.2 API), - `KNOWN_KAT_DIGEST` for the consecutive calendar.

9.6. Harness validation

The linear sampling harness is validated against a hand-computed case at w=8, r=1, α={word0 bit2, word4 bit0}, β={word0 bit3}, fixed key. Observed correlation 0.4993 (expected 0.5, within sampling noise). With random keys per sample, the observed |corr| drops to 0.0022 (expected ≈0). This confirms the harness is correct with fixed keys.

10. Verdict

MYSX-256/512 v0.2-draft is an experimental proposal, not a secure cipher. The v0.2 changes are a new rotation schedule and a new AEAD/packet format.

Rotation schedule. `rand_s9_45430` replaces the consecutive schedule `(r mod 31)+1`. On 9 measured metrics (integral at R=5 and R=6, linear at R=10 and R=14 in both directions, exact DMITM, weight-0 chain, MILP differential at R=5), the new schedule improves 6 and ties 3, with no regressions. Exact DMITM goes from 17 to 9 rounds; weight-0 chain from 13 to 5; integral balanced bits at R=5 from 21 to 9. The improvement in linear weight at R=14 (forward: 5 → 8; backward: 5 → 11) means the single-trail correlation is 2^3 and 2^6 lower than v0.1. The selection is reproducible from `search_mysx_deep.py --n 1000000 --seed 9` and documented as 31 explicit values in §2.8.

API v2. The v2 packet format (VERSION=0x02) adds a calendar selector, HKDF-SHA512 with a 64-byte secret (recovering the 2^256 Grover bound that v1 lost to

HKDF-SHA256), 512-bit MAC keys, HMAC-SHA512 tags, one key per message (32-byte salt), and Argon2id for password paths. The v1 API is kept for backwards compatibility and is marked deprecated for new data. `v2.securityLevel` reports the declared nominal security for each creds type.

PENDING for v0.2. The following analyses were measured for v0.1 and are PENDING for v0.2: the differential offset sweep and MILP upper bounds at R≥6, the linear beam beyond R=14, the exact linear hulls at r=10,11,12, the full 31-offset integral sweep and the r0=15 peak, the NIST SP 800-22 statistical battery, the truncated impossible differential (39 rounds for v0.1), the single-bit and multi-bit truncated-propagated impossibles, and the Simon normalized coverage comparison. The related-key audit is inherited (key schedule unchanged). Sections that are properties of the round function or key schedule (exact LAT of F, mask propagation, 22-test key schedule audit) are inherited unchanged.

No lower bound over 128 rounds is established, no key-recovery attack has been attempted, and MYSX v0.2 has not been externally audited. The truncated impossible differential for v0.2 is unknown; the 39-round value for v0.1 does not apply to the new schedule. The full analysis for v0.2 is incomplete by design: this draft is a working document with explicit PENDING marks and the commands needed to resolve them.

Do not use MYSX to protect real data. For production: AES-256-GCM or XChaCha20-Poly1305.

11. Reproducibility

File	Purpose
`SPEC.md`	Formal specification (identical content to §2)
`PAPER.md`	This document (v0.1-draft2 is `PAPER_v0.1.md` after tag)
`KAT.json`	v0.1 test vectors (block, AEAD, KDF)
`KAT_v02.json`	v0.2 block test vectors
`KAT_v2.json`	v0.2 API test vectors
`CALENDAR_v02.txt`	`MYSX_ROT_CAL=22,5,8,...`
`generate_kat.py`	v0.1 KAT generator
`generate_kat_v2.py`	v0.2 KAT generator (with deterministic salt/nonce)
`set_calendar.js`	Injects the calendar into `mysx.prod.js`
`verify_v2_usage.js`	12-check smoke test for v2 API
`mysx.prod.js`	Reference implementation (v1 + v2)
`mysx.test.js`	Functional tests (17 tests)
`cryptanalysis/models/linear/mysx_linear_model.py`	Exact linear model of F + round; cross-verified against `KNOWN_KAT_DIGEST`
`cryptanalysis/models/linear/LINEAR_MODEL.md`	Documentation of the exact linear model
`cryptanalysis/models/linear/linear_model_data.json`	Data behind §4 (except §4.10)

File	Purpose
`cryptanalysis/DMITM/dmitm_verify.py`	Exact DMITM search (forward + backward)
`cryptanalysis/DMITM/dmitm_trunc.py`	Truncated-bit impossible differentials
`cryptanalysis/DMITM/dmitm_impossible.py`	Miss-in-the-middle search over weight-0 chains
`cryptanalysis/DMITM/search_mysx_deep.py`	1M random schedule search
`cryptanalysis/DMITM/run_5round_test.py`	R=5 screening
`cryptanalysis/DMITM/run_full_analysis.py`	Full analysis of a candidate
`cryptanalysis/DMITM/wf2_deep_summary.json`	8 candidates × 9 metrics (raw §4.13)
`misc/mysx_linear_beam.py`	Beam search over the exact linear model
`misc/mysx_milp2.py`	Differential MILP
`misc/mysx_division_property_v2.py`	Division property
`misc/diff_free_chain_table.py`	Differential free chain table
`misc/verify_diff_trail.py`, `misc/verify_diff_13r.py`	Empirical differential verification
`.github/workflows/mysx_full_screen.yml`	WF1: 13-candidate screening
`.github/workflows/mysx_deep_dive.yml`	WF2: 8-candidate deep analysis

Reproduction commands.

v0.1 (consecutive schedule, inherited sections):

```
# No env var set: k=1 behavior
python3 misc/diff_free_chain_table.py
python3 misc/verify_diff_13r.py
python3 cryptanalysis/models/linear/mysx_linear_model.py selftest
python3 misc/mysx_linear_beam.py reproduce
```

v0.2 (new schedule):

```
export MYSX_ROT_CAL=22,5,8,7,17,10,30,1,11,26,6,3,12,23,25,13,28,20,21,19,24,2,18,9,4,1

# Exact DMITM = 9
python3 cryptanalysis/DMITM/dmitm_verify.py exact

# Weight-0 chain = 5
python3 misc/diff_free_chain_table.py

# R=10, R=14 beam weights
python3 misc/mysx_linear_beam.py run --offsets 0-30 --rounds 10 --beam 2000 --jobs 1
python3 misc/mysx_linear_beam.py run --offsets 0-30 --rounds 14 --beam 2000 --jobs 1
python3 misc/mysx_linear_beam.py run --offsets 0-30 --rounds 10 --beam 2000 --jobs 1 --
python3 misc/mysx_linear_beam.py run --offsets 0-30 --rounds 14 --beam 2000 --jobs 1 --

# Integral at r0=0
python3 misc/mysx_division_property_v2.py --rounds 5-6 --active word0 --r0 0 --solver h
```

```
# KAT cross-check
MYSX_ENABLE_TEST=1 node -e "require('./mysx.prod.js').TEST.generateKAT(1).then(console.
python3 -c "import sys;sys.path.insert(0,'cryptanalysis/models/linear');from mysx_linea
```

The Python `kat_digest()` and the JavaScript `generateKAT(1)` must produce identical outputs. This is the cross-language consistency check for the v0.2 schedule.

12. Final warning

MYSX has not been externally audited. This paper is the author's self-analysis, not a certification. No result here replaces an independent cryptographic review.

Do not use MYSX to protect real data.